

CSE-214: Structural Design Patterns

Exam Prep Pack – Templates, Recognition Guide & Practice Set

Prepared for Sami (2305055)

August 2026

Tip

This document is the companion to the 4 base templates and 5 mixed-pattern templates (.java files). It does not repeat that code – it tells you *when* to reach for which file, in how much time, and gives you 10 fresh scenarios (in the same style as A1/B1/B2/C1) to rehearse the full read → identify → template → adapt loop before next week's exam.

Contents

1	5-Minute Exam Strategy	1
2	Pattern Recognition Cheat Sheet	2
3	Disambiguation Guide	2
4	Base Templates – Quick Index	3
5	Mixed-Pattern Playbook (especially Bridge combos)	4
6	Practice Set (10 scenarios)	6
7	Answer Key – Pattern Hints Only	10

1 5-Minute Exam Strategy

Your four real quizzes ran 30–35 minutes each. That is not much time to design *and* type. The bottleneck is never "do I know the pattern" – it's translating recognition into code fast. Use this loop:

1. **Read once, mark nouns and verbs.** Nouns become classes; verbs become interface methods. Circle phrases like "*may optionally*", "*vary independently*", "*cannot change*", "*contains items or other X*" – these are the pattern's fingerprint (see Section 2).
2. **Name the pattern(s) out loud (in your head) before writing a single line.** If you're not sure between two, re-read the sentence that describes how the pieces combine – that sentence almost always settles it (see Section 3).
3. **Open the matching template file, rename top-down:** interface → leaf/concrete → decorator/adaptor/bridge classes → `main()` demo. Don't redesign the shape of the template – just rename and adjust formulas.
4. **Wire objects in `main()` in this order** when patterns are mixed: build the lowest-level concrete pieces first → compose them (Composite) → bridge them (Bridge) → decorate

last (Decorator) → adapt only where a legacy class is explicitly given (Adapter, usually first, since it produces the object everything else wraps).

5. **Check your numbers against the worked "Case" examples** in the question before you consider yourself done – every real quiz gave you these for free as a correctness check.

Careful

Don't try to memorize UML boxes. Memorize the *shape of the trick*: Decorator wraps ONE thing repeatedly; Composite holds MANY children of the same type; Adapter translates ONE incompatible class; Bridge stores ONE reference that lets TWO hierarchies multiply without new classes. Everything else is renaming.

2 Pattern Recognition Cheat Sheet

Pattern	Trigger phrases in the question	Structural signature	Cardinality
Decorator	"may optionally...", "customer may also request", "wrap", "extra charge for...", "any combination of add-ons", "adding a new add-on later shouldn't change existing code"	Wrapper implements same interface as the thing it wraps; holds exactly one wrapped reference; can nest itself	0 to many, stackable, order can matter
Composite	"contains items or groups of items", "process uniformly whether single or group", "display in hierarchical format", "nested packages/menus/folders"	Component interface with a Leaf (no children) and a Composite (holds a list of children, delegates work to them)	0 to many children, tree shape
Adapter	"legacy", "third-party", "cannot change/modify the internal code", "client is already written against interface X"	New class implements the Target interface and wraps exactly one existing Adaptee instance, translating calls	Exactly 1 incompatible class being fixed
Bridge	"vary independently", "extend both independently without modification", "the shop expects to add more X AND more Y later", "combine any Type with any Mode"	Abstraction holds a reference to an Implementor interface; both sides have their own sub-hierarchy that can be extended separately	Exactly 1 Implementor chosen per Abstraction instance, but any pairing is legal

3 Disambiguation Guide

These are the confusions that actually cost time under pressure.

Decorator vs. Bridge

Both involve one object holding a reference to another interface. The test: is the extra thing **optional and stackable** (0, 1, 2, 3... in any order), or is it a **mandatory single choice** from

a fixed set, decided once at construction? Gift-wrap on a single item = Decorator. "Must choose exactly one packaging style" = Bridge.

Composite vs. Decorator

Composite's container holds **many** children and conceptually *is made of* them (whole-part). Decorator wraps **exactly one** object and conceptually *enhances* it (it is still "the same thing, plus a feature"). If the question says "two or more items," that's Composite. If it says "a single object, optionally enhanced," that's Decorator.

Adapter vs. Bridge

Adapter is a **reactive fix**: one already-existing, already-shipped class has the wrong interface, and you patch around it. Bridge is a **proactive design**: you are building two hierarchies from scratch, and you deliberately decouple them so they can both grow later. If a legacy/third-party class is explicitly given to you as broken, that's Adapter (possibly plugged into a Bridge – see Section 5).

Adapter vs. Decorator

Both wrap one object. Adapter **changes the interface** (input type A, output type B – e.g., XML in, JSON out). Decorator **keeps the same interface** on both sides and just adds behaviour/-cost/description.

4 Base Templates – Quick Index

These four files are your starting point for a *single*-pattern question.

01_Decorator_Template.java

Component (interface: `getDescription()`, `getCost()`) → **ConcreteComponentA** (base object) → abstract **Decorator** (holds `wrapped`) → **ConcreteDecoratorA/B** (one per add-on). Rename the interface methods to whatever the question needs (price, time, description...) – the shape never changes.

02_Composite_Template.java

Component (interface) → **Leaf** (no children) → **Composite** (holds `List<Component> children`, loops over them). Comment block at the bottom shows the two most common exam twists: a *restricted* composite (some node type may only contain certain children) and a *discounted* composite (price formula isn't a plain sum).

03_Adapter_Template.java

Adaptee (existing/legacy, untouched) → **Target** (interface the client already expects, often *given* in the question) → **Adapter** (implements **Target**, wraps an **Adaptee**, translates the call).

04_Bridge_Template.java

Implementor (interface, the "how") → **ConcreteImplementorA/B** → abstract **Abstraction** (holds an **Implementor** reference, the "what") → **RefinedAbstractionX/Y**. Any **Refined**×**Concrete** pairing works with zero new classes – that's the point.

5 Mixed-Pattern Playbook (especially Bridge combos)

Real quiz questions rarely test one pattern in isolation once you're past the intro level – three of your four real questions (B1, and arguably A1, plus C1) combine two patterns. You cannot carry a template for *every* possible combination, but the five below cover what has actually appeared and what is structurally most likely to appear again.

05_Bridge_Composite_Template.java

Signature: a tree where every branch node ALSO carries a mandatory, independently-varying second attribute (e.g., "package kind" × "packaging style"). **The trick:** one class (e.g. `GiftPackage`) plays *both* roles – it implements `Node` (Composite) *and* extends `Abstraction`-style by holding a `Style implementor` field (Bridge) at the same time. **Maps to:** B1-style questions (packages that nest, plus a mandatory packaging choice that must combine freely with package kind).

06_Bridge_Decorator_Template.java

Signature: two hierarchies that must pair freely (Bridge), PLUS optional stackable extras on top of the result (Decorator). **The trick:** the Abstraction and the Decorators must implement the *same* shared interface so a Decorator can wrap either one transparently. **Build order in main():** construct the Abstraction (pick both sides of the bridge) first, decorate second.

07_Bridge_Adapter_Template.java

Signature: you're building a Bridge, but one Implementor variant is an existing/legacy class with mismatched method names. **The trick:** write an Adapter that implements `Implementor` and wraps the legacy class – it then slots into the Bridge exactly like a native `ConcreteImplementor`. The Abstraction never needs to know a translation happened.

08_Adapter_Decorator_Template.java (bonus)

Signature: exactly C1's shape – a legacy format needs adapting to a target interface, and on top of that, the client can request any combination of order-sensitive optional transforms. **The trick:** Adapter runs *first* and produces the first object satisfying `Target`; everything after that is just Decorator wrapping of `Target`. Adding a new transform later = one new Decorator subclass, zero changes elsewhere.

09_Composite_Decorator_Template.java (bonus)

Signature: a tree (Composite) where any node – single leaf OR an entire branch – can optionally get the same stackable enhancement (Decorator). **The trick:** the Decorator must implement the same interface as both `Leaf` and `Composite`, so it can wrap either without caring which. Use this instead of Bridge+Composite when the second dimension is *optional and repeatable*, not a mandatory single choice.

How your 4 real questions actually map (read this critically, not as gospel)

- **A1 (gift shop):** gift-wrap is a clean, optional, single add-on → Decorator. The delivery block is more interesting: the question explicitly says regions and modes should each be extensible "*independently*" – that is textbook Bridge language (Region = Implementor, Mode = Abstraction, or vice versa), with the whole delivery block then attached to the item the same way a Decorator attaches gift-wrap. Don't assume "one item, a few extra charges" always means pure Decorator – check whether the extras themselves form two independently-growing families first.

- **B1 (gift packages):** the nesting ("packages of packages") is unambiguous Composite. The packaging-style choice is genuinely debatable: it's a *mandatory single choice* that must combine freely with package kind (2 kinds $\times$ 3 styles, no new class per pair) – which is the exact scenario Bridge exists for, and is the reading used in template 05. A Decorator-only reading (treat style as a single optional wrap) also compiles and often earns partial/full credit in practice – but it doesn't capture "must choose exactly one" as cleanly. If your exam question reads this way, lead with Bridge, and mention the Decorator alternative in a comment if you have time.
- **B2 (food order):** pure Composite, with two twists worth memorizing: a *restricted* composite (Set Menu can't nest another Set Menu or groceries) and a *non-additive* composite (Set Menu price is items minus 10%). Both are one-line overrides on top of the base template.
- **C1 (XML/JSON API):** unambiguous Adapter (XML $\rightarrow$ JSON, legacy code untouched) + Decorator (encrypt/compress, any combination, order-sensitive, easy to add a third transform later).

6 Practice Set (10 scenarios)

Same format and difficulty band as your real quizzes. Time yourself. Solve using the templates – don't write from scratch. An answer key (pattern only, no code) is in Section 7 – don't peek until you've attempted each one.

P1 – Ride-Share Fare Calculator

Suggested time: 25 minutes

A ride-hailing app charges a base fare of \$2.00 plus \$1.50 per kilometre travelled. Riders may optionally add any combination of:

- **Pet Fee** – adds \$3.00 (travelling with a pet)
- **Extra Luggage** – adds \$2.00 per extra bag
- **Priority Pickup** – adds \$5.00 (guarantees pickup within 5 minutes)

The platform plans to add more optional add-ons later (child seat fee, toll pass, etc.) and adding one must not require changing existing fare-calculation classes. Implement the fare calculator.

Case 1: 4 km ride, no add-ons $\Rightarrow$ Total = \$2.00 + (4 $\times$ \$1.50) = **\$8.00**.

Case 2: 6 km ride with a pet and 2 extra bags $\Rightarrow$ Total = \$2.00 + (6 $\times$ \$1.50) + \$3.00 + (2 $\times$ \$2.00) = **\$18.00**.

Case 3: 10 km ride with Priority Pickup and a pet $\Rightarrow$ Total = \$2.00 + (10 $\times$ \$1.50) + \$5.00 + \$3.00 = **\$25.00**.

P2 – File System Size Report

Suggested time: 25 minutes

Model a simplified file system. A **File** has a name and a size in KB. A **Folder** can contain files and/or other folders (any depth). Implement:

1. Computing the total size of a folder, recursively.
2. Printing the folder's contents in an indented, hierarchical listing.

The system should support new node types later (e.g., symbolic links) without modifying existing classes.

Case: Root contains A.txt (120 KB) and a folder Docs, which contains B.pdf (300 KB) and a folder Sub, which contains C.txt (50 KB). Total size of Root = **470 KB**.

P3 – Legacy Payment Gateway

Suggested time: 25 minutes

Your checkout code is written against a modern interface:

```
interface PaymentProcessor {
    boolean pay(double amountInDollars);
}
```

The company already has a working integration with a third-party library that cannot be modified (maintained externally):

```
class OldMoneyGateway {
    // amount must be given in CENTS, as a String; returns 0 for success,
    // 1 for failure
    int sendPayment(String amountInCents) { ... }
```

```
}

```

Make `OldMoneyGateway` usable anywhere `PaymentProcessor` is expected, without changing either class.

Case: paying \$45.50 must call `sendPayment("4550")`; a return value of 0 must translate to `pay(...)` returning `true`.

P4 – Cross-Platform Media Player

Suggested time: 30 minutes

A media-player app plays different media **types** (`AudioTrack`, `VideoClip`) on different **platforms** (`Windows`, `Mac`, `LinuxEmbedded`), where each platform has its own low-level decode routine. The company plans to add more media types AND more platforms over time, and extending one must not require touching the other hierarchy. Each platform defines a fixed decode overhead (ms); each media type defines a multiplier applied to that overhead. Platform overheads: `Windows` = 10ms, `Mac` = 20ms, `LinuxEmbedded` = 50ms. Type multipliers: `AudioTrack` = 1×, `VideoClip` = 4×.

Case 1: `VideoClip` on `Windows` $\Rightarrow 10 \times 4 = \mathbf{40\ ms}$ buffering.

Case 2: `AudioTrack` on `LinuxEmbedded` $\Rightarrow 50 \times 1 = \mathbf{50\ ms}$ buffering.

P5 – Custom Furniture Sets (*Bridge + Composite*)

Suggested time: 35 minutes

A furniture store sells individual pieces (chair, lamp, table, ...), each with a base price. Customers can build custom **Furniture Sets** out of two or more pieces and/or other sets (a set may contain another set). Every set is one of two **kinds**: `LivingRoomSet` or `OfficeSet` (this only affects the printed category label). Independently, every set – regardless of kind – must be finished in *exactly one* of three finishes: `OakFinish` (+\$25), `WalnutFinish` (+\$60), `PaintedFinish` (+\$15). Kind and finish must combine freely (2 kinds $\times$ 3 finishes with no class per combination), and the store plans to add more kinds and finishes later, independently.

Case: a `LivingRoomSet` with `WalnutFinish` containing a Chair (\$80) and a Lamp (\$40) $\Rightarrow$ Total = \$80 + \$40 + \$60 = **\$180**.

P6 – Smart-Home Notification Hub (*Bridge + Decorator*)

Suggested time: 30 minutes

A smart-home hub sends alerts of different **types** (`SecurityAlert`, `MaintenanceAlert`) over different **channels** (`PushChannel` base \$1.00, `VoiceChannel` base \$4.00). New alert types and new channels will be added independently over time. `SecurityAlert` adds a \$0.50 urgency surcharge over the channel's base cost; `MaintenanceAlert` adds nothing. On top of any alert, the user may optionally request **Priority Flag** (+\$0.50) and/or **Delivery Confirmation** (+\$0.20), in any combination, without modifying the alert or channel classes. A third optional feature may be added later.

Case 1: `MaintenanceAlert` via `PushChannel`, no extras $\Rightarrow$ Total = **\$1.00**.

Case 2: `SecurityAlert` via `VoiceChannel` with Priority Flag and Delivery Confirmation $\Rightarrow$ Total = \$4.00 + \$0.50 + \$0.50 + \$0.20 = **\$5.20**.

P7 – IoT Sensor Integration (*Bridge + Adapter*)

Suggested time: 30 minutes

A home-automation system reads sensors through a standard company interface:

```
interface SensorDriver {
    double read();
}
```

Different **monitor types** (`TemperatureMonitor`, `HumidityMonitor`) each hold a `SensorDriver` and can be paired with any driver, including future ones. The company just acquired a hardware partner whose devices ship with a vendor-locked, unmodifiable class:

```
class LegacySensorLib {
    // returns raw value in tenths of a unit, e.g. 235 means 23.5
    int fetchRawValue() { ... }
}
```

Integrate `LegacySensorLib` into the monitoring system *without* changing `TemperatureMonitor`/`HumidityMonitor` or the `SensorDriver` interface.

Case: `LegacySensorLib.fetchRawValue()` returns 235 $\Rightarrow$ a `TemperatureMonitor` wired to the adapted driver must report **23.5**.

P8 – Multi-Format Report Export (*Adapter + Decorator*)

Suggested time: 30 minutes

An internal reporting engine only knows how to produce a report via a fixed, legacy method that cannot be changed:

```
class LegacyReportEngine {
    String getReportAsCSV() { ... }
}
```

A third-party dashboard consumes reports through:

```
interface Report {
    String render();
}
```

Per request, the caller may ask for the rendered output to be **Uppercased** and/or wrapped with a **Timestamp Header**, in either order (order affects the result, since uppercasing after adding the header also capitalizes the header text). A third optional post-processing step (e.g., a checksum footer) may be added later without touching `LegacyReportEngine`, the adapter, or the existing wrappers.

P9 – Nested Document Builder (*Composite + Decorator*)

Suggested time: 30 minutes

A document tool builds documents from **Sections**. A **Section** contains **Paragraphs** (leaf, has a word count) and/or other **Sections** (any depth). Implement recursive total word count and an indented structure printout. Additionally, *any* node – a single paragraph or an entire section – can optionally be wrapped with a **Highlight** marker (visual marker only, word count unchanged) and/or a **Track-Changes** marker (appends "[edited]" to its printed label, word count unchanged), in any combination, without modifying **Paragraph** or **Section**.

Case: Section "Introduction" contains Paragraph P1 (120 words) and Paragraph P2 (80 words). The root document also directly contains Paragraph P3 (150 words). Total word count = 120 + 80 + 150 = **350 words**, unchanged whether or not Introduction is Highlighted or P3 is marked Track-Changes.

P10 – Print Shop Order System (Bonus / Advanced, optional)

Suggested time: 40 minutes – attempt only if time permits

A print shop processes **PrintJobs** (a single document: page count + a per-page rate depending on paper quality) that can be grouped into **PrintBundles**, which may contain jobs and/or other bundles (any depth) – *Composite*. Every bundle is one of two **purposes**: **PersonalBundle** or **BusinessBundle** (label only), and must also be assigned *exactly one* **paper quality**, which sets the per-page rate: **Standard** (\$0.05/page), **Glossy** (\$0.10/page), **Recycled** (\$0.04/page). Purpose and paper quality must combine freely, and both lists will grow independently over time – *Bridge*. Finally, any single job or entire bundle can optionally receive **Lamination** (+\$1.00/page, computed on that node's own page count) and/or **Spiral Binding** (+\$3.00 flat), in any combination and order, added later without touching existing classes – *Decorator*.

Case: a **BusinessBundle** (Glossy) contains: job "Report" (50 pages, Laminated) and job "Cover" (5 pages, plain). The whole bundle is also Spiral-Bound.

- Report: $50 \times \$0.10 = \5.00 , + Lamination ($50 \times \$1.00 = \50.00) = \$55.00
- Cover: $5 \times \$0.10 = \0.50
- Bundle subtotal = $\$55.00 + \$0.50 = \$55.50$
- + Spiral Binding on the bundle = $\$55.50 + \$3.00 = \mathbf{\$58.50}$

Tip

This one genuinely needs all three templates at once: a class plays Composite + Bridge simultaneously (like **GiftPackage** in template 05), and Decorators wrap either a bare **PrintJob** or a whole **PrintBundle** on top (like template 09). Start from 05, then layer in the Decorator base/subclasses from 09.

7 Answer Key – Pattern Hints Only

No code here on purpose – these are for checking your *identification*, not for copying. If you disagree with one after solving it, that's worth sitting with: these questions were written to have a clearly best-fit pattern, but exam wording is never perfectly unambiguous, and part of the skill is defending your reading.

#	Pattern(s)	Why
P1	Decorator	Single base fare, any number of optional stackable add-ons, new add-ons shouldn't touch existing code.
P2	Composite	Folders contain files and/or folders, uniform recursive processing.
P3	Adapter	One incompatible, unmodifiable legacy class; client already coded against PaymentProcessor .
P4	Bridge	Media type and platform must each extend independently; no per-combination class.
P5	Bridge + Composite	Sets nest (Composite); kind $\times$ finish must combine freely as a mandatory single choice each (Bridge) – same shape as B1.
P6	Bridge + Decorator	Alert type $\times$ channel vary independently (Bridge); Priority/Confirmation are optional and stackable on top (Decorator).
P7	Bridge + Adapter	Monitor type $\times$ driver is a Bridge; the vendor-locked class needs an Adapter to satisfy SensorDriver before it can be plugged in.
P8	Adapter + Decorator	CSV $\rightarrow$ Report is Adapter; Uppercase/Timestamp are optional, order-sensitive, stackable transforms – same shape as C1.
P9	Composite + Decorator	Sections/Paragraphs nest (Composite); Highlight/Track-Changes are optional stackable markers on any node, not tied to node type (Decorator).
P10	Composite + Bridge + Decorator	Bundles nest (Composite); purpose $\times$ paper quality is a mandatory independent pairing (Bridge); Lamination/Binding are optional stackable extras on any node (Decorator).

Careful

If you're still unsure after checking the key, re-read the disambiguation rules in Section 3 against the exact sentence that describes how the two things combine – "must choose exactly one" vs. "may optionally add" is almost always the deciding sentence.